

Java I/O e NIO.2

Definizione: cos'è l'I/O e la distinzione byte vs caratteri (il pacchetto java.io)

I/O (Input/Output) è il meccanismo con cui un programma Java legge e scrive dati verso l'esterno: console, tastiera, file su disco. Il concetto fondamentale è che **Java lavora in due modalità diverse**:

- **Stream (flussi) di byte** → per **dati binari** (immagini, file eseguibili, .class). Le classi hanno la parola Stream nel nome: FileInputStream, FileOutputStream, ObjectInputStream, ObjectOutputStream.
- **Reader e Writer** → per **dati di testo** (caratteri e stringhe). Classi come FileReader, BufferedReader, FileWriter, BufferedWriter, PrintWriter.

La regola mnemonica che dà il docente: se il nome contiene **Stream** → byte; se contiene **Reader/Writer** → caratteri.

La gerarchia delle classi e il pattern "wrapping"

Alla base ci sono **quattro classi astratte**:

Astratta	Tipo di dato
InputStream	byte (lettura)
OutputStream	byte (scrittura)
Reader	caratteri (lettura)
Writer	caratteri (scrittura)

Da queste derivano due "livelli":

- **Classi di basso livello**: si ottengono antepoendo File al nome astratto → FileInputStream, FileOutputStream, FileReader, FileWriter. Accedono direttamente al file.
- **Classi di alto livello (efficienza/buffering)**: si ottengono antepoendo Buffered → BufferedInputStream, BufferedOutputStream, BufferedReader, BufferedWriter.

Il meccanismo chiave è il **wrapping** (incapsulamento): una classe di alto livello "avvolge" una di basso livello per aggiungerle funzionalità (tipicamente il buffering, per efficienza). Esempio dal corso:

```
// Wrapping: BufferedReader avvolge FileReader
```

```
BufferedReader reader = new BufferedReader(new FileReader("source.txt"));
```

```
// Wrapping per la scrittura di oggetti
```

```
ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("file.ser"));
```

Ci sono anche **classi "ponte"**:

- `InputStreamReader` → ponte da flusso di **byte** a flusso di **caratteri**.
- `OutputStreamWriter` → ponte inverso, da **caratteri** a **byte**.

Infine: **`System.in`** rappresenta la tastiera, **`System.out`** la console.

Esempio: copia di un file di testo (con e senza buffering)

Il docente mostra un metodo che copia un file di testo, dove un booleano decide se usare o no il buffering. Usa il costrutto **`try-with-resources`**, che chiude automaticamente le risorse e **svuota (flush) i buffer sul disco** — importante perché dati "in attesa" nel buffer verrebbero altrimenti persi se il programma si bloccasse.

Con buffering — si usa `readLine()`, esclusivo di `BufferedReader`, che legge un'intera riga in una volta:

```
try (var reader = new BufferedReader(new FileReader(source));
    var wtr = new BufferedWriter(new FileWriter(dest))) {
    String str = null;
    while ((str = reader.readLine()) != null) { // null = fine dello stream/file
        wtr.write(str);
        wtr.newLine(); // readLine() elimina il fine-riga, quindi va reinserito
    }
}
```

Punti chiave spiegati:

- `readLine()` restituisce **`null`** quando si arriva alla fine del file.
- `readLine()` **elimina** il carattere di fine riga, quindi bisogna riaggiungerlo con `newLine()`, altrimenti il file di destinazione sarebbe una sola riga orizzontale.

Senza buffering — si legge **un byte alla volta**:

```
int b;
while ((b = reader.read()) != -1) { // -1 segnala la fine dello stream
    wtr.write(b);
}
```

Perché il tipo è `int` e non `byte`? Perché Java usa il valore **-1** per segnalare la fine dello stream. Tutti i caratteri validi (ASCII/Unicode) sono valori non negativi, quindi **-1** è riservato univocamente a "fine file".

Esempio: copia di un file binario + percorsi assoluti/relativi

Per i file binari (es. un `.class`) si usano gli **Stream** invece di `Reader/Writer`:

```
try (var in = new BufferedInputStream(new FileInputStream(source));
    var out = new BufferedOutputStream(new FileOutputStream(dest))) {
```

```
byte[] buffer = new byte[1024]; // buffer da 1 KB

int bytesRead;

while ((bytesRead = in.read(buffer)) > 0) {
    out.write(buffer, 0, bytesRead);
}
}
```

Il concetto importante è **perché serve bytesRead**. Un file raramente è un multiplo esatto di 1024 byte. Esempio del docente: un file di **2058 byte**. Alla prima iterazione legge 1024, alla seconda altri 1024 (totale 2048), alla terza **solo 10 byte**. Se scrivessi sempre 1024 byte fissi, l'ultimo blocco conterrebbe spazzatura. `write(buffer, 0, bytesRead)` scrive esattamente quanti byte sono stati effettivamente letti.

Sui percorsi (path):

- **Backslash ** (Windows) va "escapato": si scrive `\\`. Se scrivi `\U`, Java lo interpreta come sequenza di escape non valida e dà errore. `\\` dice a Java "è un singolo backslash letterale".
- **Forward slash /** invece **non** va escapato.
- **Percorso assoluto**: il percorso completo dalla radice (es. `C:\...`).
- **Percorso relativo**: usa il punto `.` per indicare la directory corrente. La directory di lavoro si ottiene con `System.getProperty("user.dir")`.

La classe Console

Definizione: `Console` è una classe progettata specificamente per interagire con l'utente da riga di comando. È un **singleton**: ha tutti i costruttori privati e si ottiene tramite un **factory method**, `System.console()` (nota: **non** si usa `new`).

```
Console console = System.console();

if (console == null) {
    System.err.println("Console non disponibile");
} else {
    String name = console.readLine("Inserisci il nome %s: ", name);
    console.format("Hello there %s%n", name);
    console.printf("Welcome", ...); // printf usa varargs, 0 argomenti è valido
    console.writer().println();    // riga vuota
}
```

Attenzione: il controllo `if (console == null)` è necessario perché **spesso gli IDE si "impossessano" della console e non la restituiscono** — per questo l'esempio va eseguito da riga di comando reale.

I segnaposto delle stringhe formattate:

- %S → inserisce una stringa
- %n → inserisce un ritorno a capo/nuova riga
- %d → inserisce un numero intero

Metodo `readPassword()` — molto importante per la sicurezza:

```
char[] password = console.readPassword("Password tra %d e %d caratteri: ", 4, 10);
char[] verify = console.readPassword("Verifica la password");
if (Arrays.equals(password, verify)) {
    console.printf("Le password corrispondono");
} else {
    console.printf("Le password non corrispondono");
}
```

Perché restituisce un **`char[]`** e non una **`String`**? Perché una **`String`** potrebbe restarsene nel **pool delle stringhe** in memoria, dove un hacker potrebbe raggiungerla. Con un array di caratteri questo non accade (lo si può sovrascrivere). Inoltre **`readPassword()`** **non fa eco** dei caratteri mentre digiti — non li mostra a schermo.

L'esecuzione da riga di comando usa `java -cp <classpath> <package.Classe>`, dove `-cp` imposta il classpath (la radice della gerarchia dei package).

NIO.2 (il pacchetto `java.nio.file`)

Definizione: cos'è NIO.2 e perché sostituisce `java.io.File`

NIO.2 è un'API introdotta in **Java 7** (nei pacchetti `java.nio.file` e `java.nio.file.attribute`), molto più potente della vecchia classe `java.io.File`. È il modo moderno di lavorare con file e directory. I tre elementi centrali:

- **Path** (interfaccia): rappresenta un file o una directory. **Sostituisce `File`**. Gli oggetti `Path` sono **immutabili**.
- **Paths** (classe di utilità): contiene metodi **statici** per **creare** oggetti `Path`.
- **Files** (classe di utilità): contiene metodi **statici** per **operare** sugli oggetti `Path` (li prende come parametri).

Creare un `Path`: `Path.of` vs `Paths.get`

Due stili equivalenti:

- **`Path.of(...)`** — introdotto in **Java 11**, stile nuovo.
- **`Paths.get(...)`** — stile tradizionale.

Entrambi sono **sovraccaricati per accettare varargs**:

```
// Come singola stringa
```

```
Path p1 = Path.of("logo/logo.png");
Path p2 = Paths.get("logo/logo.png");
```

// Come varargs (Java costruisce la stringa unendoli)

```
Path p3 = Path.of("logo", "logo.png");
Path p4 = Paths.get("logo", "logo.png");
```

Classificazione dei percorsi negli esempi:

- **Relativo:** nessun backslash all'inizio → relativo alla directory corrente. Es. "NI02/source.txt".
- **Assoluto Windows:** inizia con la radice C:\. Es. "C:\\Users\\...".
- **Assoluto Linux/Mac:** inizia con / (backslash all'inizio nell'esempio).

Concetto fondamentale: **questi sono percorsi logici**. Il file **non deve esistere** sul disco perché l'oggetto Path venga creato con successo. Stai solo manipolando stringhe.

2.3 Operazioni di ispezione su un Path

Dato un percorso reale come C :

\\Users\\sKennedy\\Documents\\NetBeansProjects\\Udemy\\pom.xml:

```
Path path = Paths.get("C:\\Users\\sKennedy\\Documents\\NetBeansProjects\\Udemy\\pom.xml");

path.toString();    // stampa il percorso completo
path.getNameCount(); // 6 → numero di "elementi del nome" (la radice NON conta)
path.getName(0);    // Users
path.getName(1);    // sKennedy ... e così via
path.getFileName(); // pom.xml → l'ultima parte (dir o file)
path.getParent();   // tutto tranne pom.xml
path.getRoot();     // C:\
path.isAbsolute();  // true (parte da C:\)
```

Regole importanti:

- La **radice non è un "name element"**. Gli indici partono da 0 = primo elemento *dopo* la radice (Users = indice 0).
- **getName(0)** su un percorso costituito dalla sola radice lancia **IllegalArgumentException**.
- Su un **percorso relativo**, **getRoot()** restituisce **null**.

subpath(begin, end) — restituisce un sotto-percorso. Il numero di elementi è end - begin, partendo dall'indice begin, **senza mai partire dalla radice**:

```
path.subpath(0, 3); // 3 elementi da indice 0: Users, sKennedy, Documents
path.subpath(1, 4); // 3 elementi da indice 1: sKennedy, Documents, NetBeansProjects
path.subpath(2, 3); // 1 elemento da indice 2: Documents
```

Operazioni sui file con `Files`: copia, spostamento, cancellazione

```
Path source = Paths.get("NIO2/source.txt");
Path target = Paths.get("NIO2/target.txt");
Path target2 = Paths.get("NIO2/target2.txt");
```

```
Files.createDirectories(Paths.get("NIO2")); // crea la dir e tutte le sotto-dir necessarie
Files.exists(source); // controlla se esiste
```

```
Files.createFile(source); // crea un file vuoto sul disco
Files.copy(source, target); // copia
Files.move(target, target2); // sposta = come una rinomina
Files.delete(target2); // cancella
```

Distinzioni importanti:

- **createDirectories** (plurale) crea la directory **e tutte le sotto-directory** mancanti; **createDirectory** (singolare) crea solo una directory.
- **Files.move** funziona come una **rinomina**.
- **StandardCopyOption.REPLACE_EXISTING**: passato a `Files.copy`, sovrascrive il file di destinazione se esiste, invece di lanciare eccezione. Senza di esso, se il target esiste → eccezione.

```
Files.copy(source, target, StandardCopyOption.REPLACE_EXISTING);
```

- **Copia superficiale vs profonda**: se copi una **directory**, `Files.copy` fa una **shallow copy** (copia superficiale) — i file e le sotto-directory al suo interno **non** vengono copiati. Per una **deep copy** (l'intero albero) servono gli stream (esempio trattato più avanti nel corso).

Concatenare percorsi: **resolve**

Definizione: per concatenare due `Path` si usa il metodo **resolve**. Regola mnemonica del docente: "**Resolve = Concatena**" (RC).

```
Path absolute = Paths.get("C:\\Users\\sKennedy\\Documents\\NetBeansProjects\\Udemy");
Path relative = Paths.get("NIO2");
Path file = Paths.get("name.txt");
```

```
absolute.resolve(relative); // ...\\Udemy\\NIO2 (aggiunge il relativo all'assoluto)
```

```
absolute.resolve(file);    // ...\Udemy\name.txt
```

```
relative.resolve(file);    // NIO2\name.txt (Java mette il backslash in mezzo)
```

Regole fondamentali di `resolve`:

- **Non controlla** se i percorsi esistono sul disco: lavora **solo sulle stringhe**.
- **Se l'argomento passato è un percorso assoluto, viene restituito quel percorso assoluto e basta** — qualunque sia il percorso su cui chiami `resolve`. Esempio:
`relative.resolve(absolute)` → restituisce semplicemente `absolute`.

```
relative.resolve(absolute); // restituisce absolute
```

```
file.resolve(absolute);    // restituisce absolute
```

- Poiché lavora sulle stringhe, non gli interessa se un nome "sembra" un file o una directory. Se hai `dir` (che potrebbe essere un file) e `sk.txt` (che potrebbe essere una directory), `p1.resolve(p2)` semplicemente li concatena.

La serializzazione

(Il docente prosegue dentro la sezione *NIO.2*, ma qui cambia argomento verso la serializzazione.)

Definizione

Serializzazione = trasformare un oggetto in una sequenza di byte per salvarlo su file (o trasmetterlo). **Deserializzazione** = il processo inverso, ricostruire l'oggetto dai byte. Si usa `ObjectOutputStream.writeObject()` per serializzare e `ObjectInputStream.readObject()` per deserializzare. La classe deve implementare l'interfaccia **Serializable**.

// Serializzazione

```
try (var out = new ObjectOutputStream(
    new BufferedOutputStream(
        new FileOutputStream("book.ser")))) {
    Book book = new Book();
    book.setAuthor("Sean Kennedy");
    out.writeObject(book);
}
```

// Deserializzazione

```
try (var in = new ObjectInputStream(
    new BufferedInputStream(
        new FileInputStream("book.ser")))) {
```

```
Book b = (Book) in.readObject(); // downcast necessario
System.out.println(b);
}
```

NotSerializableException e transient

Se un oggetto contiene un campo non serializzabile (es. un tipo `Image`), `writeObject` lancia **NotSerializableException**. Due soluzioni:

1. **Rendere serializzabile** anche quel tipo.
2. Marcare il campo come **transient**: così quel campo **non viene serializzato** affatto.

```
private transient Image image; // non verrà serializzato
```

I costruttore NON viene chiamato in deserializzazione

Concetto d'esame fondamentale. Nell'esempio, il costruttore di `Book` imposta l'autore a "unknown", ma prima di serializzare viene cambiato in "Sean Kennedy". Alla deserializzazione si riottiene "Sean Kennedy", non "unknown":

Se la classe è serializzabile, il suo costruttore NON viene chiamato durante la deserializzazione.

Ma con l'ereditarietà cambia: se il **genitore NON è serializzabile**, allora il **costruttore del genitore VIENE chiamato** durante la deserializzazione.

Esempio: `Book extends InfoMedium implements Serializable`, dove `InfoMedium` **non** è serializzabile:

```
class InfoMedium { // NON serializzabile
    String medium;
    InfoMedium() { medium = "unknown"; } // QUESTO costruttore viene chiamato
}

class Book extends InfoMedium implements Serializable {
    String author;
    Book() { author = "unknown"; } // questo NON viene chiamato
}
```

Risultato: dopo aver serializzato `medium="print"`, `author="Sean Kennedy"`:

- **prima** → "print Sean Kennedy"
- **dopo (deserializzato)** → "unknown Sean Kennedy" (il `medium` è tornato a "unknown" perché il costruttore del genitore non-serializzabile è stato eseguito).

Nota collaterale: un campo `transient` di tipo riferimento (es. `String`) alla deserializzazione riprende il **valore di default del tipo**, cioè `null`.

3.4 serialVersionUID e InvalidClassException

Se la **definizione della classe** cambia rispetto a quella salvata su file (es. aggiungi un campo), la deserializzazione lancia **InvalidClassException**, perché i `serialVersionUID` non corrispondono. Se non ne dichiari uno, Java ne genera uno automaticamente in background; il `serialVersionUID` serve proprio a **verificare che la definizione della classe corrisponda** a quella salvata sul file esterno.

3.5 Serializzazione personalizzata: `writeObject` / `readObject` e `ObjectStreamField`

Puoi inserire **codice personalizzato** nel processo (utile, ad esempio, per **cifrare** dati sensibili prima di salvarli e **decifrarli** al ritorno).

L'array `ObjectStreamField[] serialPersistentFields` specifica quali campi serializzare — è come una **whitelist**. Il rovescio: un campo `transient` è come una **blacklist** (non salvato). Se un campo (es. `authorAge`) **non** è nell'array, di fatto è come se fosse `transient` e non viene scritto.

```
private static final ObjectStreamField[] serialPersistentFields = {  
    new ObjectStreamField("author", String.class),  
    new ObjectStreamField("isbn", String.class)  
    // authorAge NON è qui → non verrà serializzato  
};  
  
private void writeObject(ObjectOutputStream oos) throws IOException {  
    ObjectOutputStream.PutField fields = oos.putFields();  
    fields.put("author", author);  
    fields.put("isbn", isbn);  
    oos.writeFields();  
}  
  
private void readObject(ObjectInputStream ois) throws Exception {  
    ObjectInputStream.GetField fields = ois.readFields();  
    author = (String) fields.get("author", null); // null = valore se assente  
    isbn = (String) fields.get("isbn", null);  
}
```

Nel test, `authorAge = 35` non viene serializzato, quindi al ritorno vale **0** (default di un `int`). La cifratura andrebbe fatta nel `writeObject` (righe del `put`) e la decifratura nel `readObject` (righe del `get`).

Local Variable Type Inference (var)

4.1 Definizione

Java è un linguaggio **fortemente tipizzato**: normalmente devi dichiarare il tipo di ogni variabile. Da **Java 10** puoi usare la parola chiave **var** in certe situazioni: il **compilatore inferisce** il tipo dal valore assegnato.

```
var x = 2;    // int
var y = 3.2;  // double
var t = "ABC"; // String
```

Le 7 regole sulle var

1. **Dove si possono usare:** solo come **variabili locali**, cioè dentro blocchi di codice: costruttori, metodi, blocchi di inizializzazione d'istanza.
2. **Dove NON si possono usare:** come parametri di costruttore, parametri di metodo, o variabili static di classe / variabili d'istanza.
 - **Eccezione (caveat):** nelle **lambda**, dove il tipo del parametro può essere inferito. Esempio con `removeIf` su una `List<String>`: Java sa che l'elemento è una `String`, quindi `var` è lecita lì.
3. **Inizializzazione obbligatoria nella stessa istruzione** della dichiarazione. Non puoi fare `var x;` e poi `x = 2;` su due righe → errore *"cannot infer type for local variable"*.
4. **Il valore può cambiare, ma il tipo no:**

```
var d = 4.5;    // double
d = 98.9;       // OK, valore cambiato
d = "SSS";      // ERRORE: il tipo resta double
```

5. **Non può essere inizializzata a null direttamente** (il compilatore non saprebbe che tipo inferire). Devi prima fare il cast:

```
var s = (String) null; // OK
```

6. **Non permessa nelle dichiarazioni multi-variabile:**

```
int a = 3, b = 4;    // OK
var a = 3, b = 4;    // ERRORE
```

7. **var è un nome riservato ma non una parola riservata.** Quindi puoi usarlo come **identificatore**: `var var = 8;` è valido. Ma non puoi chiamare `var` una classe, un'interfaccia o un enum.
-

Metodi privati nelle interfacce

5.1 Definizione

Introdotti in **Java 9**: le interfacce possono avere **metodi privati**, sia **static** che **non-static**. Essendo privati, sono accessibili **solo all'interno dell'interfaccia**. Come nelle classi, **non puoi chiamare un metodo non-static da un contesto static**.

Hanno **due vantaggi**:

1. **Ridurre la duplicazione del codice.**
2. **Migliorare l'incapsulamento**, nascondendo i dettagli implementativi.

5.2 Esempio: da `InefficientTennis` a `Tennis`

Nella versione inefficiente, tre metodi (`forehand`, `backhand`, `smash`) ripetono ognuno le stesse righe ("spostati in posizione", "torna in posizione di pronto"). La versione efficiente estrae la logica comune in un **metodo privato**:

```
interface Tennis {  
  
    private static void hit(String stroke) {  
        System.out.print("spostati in posizione");  
        System.out.print("colpisci un " + stroke);  
        System.out.print("torna in posizione di pronto");  
    }  
  
    static void forehand() { hit("forehand"); }  
    default void backhand() { hit("backhand"); }  
    private void smash() { hit("smash"); } // metodo privato non-static  
}
```

Punti chiave:

- Il metodo privato `hit` può essere chiamato da un contesto **static**, da un metodo **default** e da un altro metodo **privato**, tutti senza problemi (siamo dentro l'interfaccia). → risolve la **duplicazione**.
- Dall'esterno, un riferimento `Tennis tennis = new ProfessionalTennis();` **non** può accedere a `hit()`: è privato, incapsulato. → risolve l'**incapsulamento**.

Regole ricordate a margine:

- Da un metodo **static** puoi chiamare solo altri metodi **static** (`static` → `static`). Un metodo d'istanza richiede un'istanza.
 - Un metodo **abstract** (di default nelle interfacce) **non può avere un corpo**.
 - I modificatori **private** e **default** **non possono coesistere**: significano cose opposte — `default` è pensato per i tipi che implementano l'interfaccia, `private` significa privato all'interfaccia.
-

Le annotazioni

6.1 Definizione: metadati e annotazioni

Un **metadato** è "informazione sull'informazione". Analogia del docente con un database: il dato è "Sean Kennedy", il **nome della colonna** (name) è un metadato che ti dice che quel dato è un nome.

Le **annotazioni** permettono, tramite i metadati, di **aggiungere valore al codice**. Puoi annotare classi, metodi, variabili, ecc. Caratteristiche:

- Funzionano molto **come le interfacce** (le loro origini sono nelle interfacce); i loro elementi assomigliano a metodi abstract.
- I nomi sono **case sensitive**; per convenzione iniziano con la **maiuscola** (es. `@Override`).
- Iniziano con `@` seguito dal nome.
- Come le interfacce, possono essere applicate a classi non correlate.

6.2 `@Override` e `@FunctionalInterface`

`@Override` segnala che stai sovrascrivendo un metodo del genitore. Vantaggio pratico: se sbagli a scriverlo (es. `toString` con la "s" minuscola invece di `toString`), il compilatore **individua l'errore** grazie a `@Override`. Senza, sarebbe solo "un altro metodo" e l'output resterebbe quello poco leggibile ereditato da `Object` (package + classe + `@` + hash code).

```
@Override
```

```
public String toString() { return name; }
```

`@FunctionalInterface` impone la **SAM rule** (Single Abstract Method): è ammesso **un solo metodo abstract**. Se ne aggiungi un secondo → errore di compilazione. Punto d'esame importante:

I metodi ereditati da `Object` (come `equals`) NON contano ai fini della SAM rule.

Quindi un'interfaccia con un solo metodo astratto proprio + un `equals` è comunque una valida functional interface.

6.3 `@Deprecated`, `@SuppressWarnings`, `@SafeVarargs`

`@Deprecated` marca un metodo come obsoleto (da non usare più). Ha due elementi opzionali:

- **`since`** → da quale versione è deprecato.
- **`forRemoval`** → `true` se è previsto per la rimozione futura.

```
@Deprecated(since = "2.0", forRemoval = true)
```

```
public static void print() { ... }
```

Va accompagnato da un commento **Javadoc `@deprecated`** (in Javadoc è **tutto minuscolo**, mentre l'annotazione ha l'iniziale maiuscola). L'IDE mostra il metodo **barrato**. Non blocca l'esecuzione, genera solo un **warning** del compilatore. (Esempio reale citato: il costruttore di `Integer` è deprecato in favore di `Integer.valueOf`, che usa un pool di interi per risparmiare memoria.)

`@SuppressWarnings` dice "non voglio quell'avviso":

- `@SuppressWarnings("deprecation")` → sopprime gli avvisi di deprecazione.
(Nota del docente: per la deprecazione a volte l'avviso spunta comunque — pare un bug noto del compilatore, discusso su *Stack Overflow*.)
- `@SuppressWarnings("unchecked")` → sopprime gli avvisi legati ai **raw type**.
Esempio: usare `new ArrayList()` (parentesi vuote, raw type) invece di `new ArrayList<>()` (operatore diamond) genera l'avviso *"uses unchecked or unsafe operations"*; l'annotazione lo silenzia.

@SafeVarargs si mette davanti a un metodo per dichiarare che ti comporterai correttamente con i varargs in arrivo. Il docente mostra un contro-esempio (**abuseVarargs**) che **abusa** dei varargs e causa una **ClassCastException**: prende una lista di `Integer` come vararg, fa puntare un array di `Object` a quella lista, ci mette dentro una lista di `String`, e poi `list.get(0).get(0)` fallisce perché tenta di passare da `String` a `int`.

Regole su **@SafeVarargs**:

- Devi **avere i varargs** (i tre puntini `...`), altrimenti errore.
- Il metodo deve essere **private, static oppure final**.